

Code Reading for Gate - Map Literacy

You do not need to be a software engineer yet. You need to read the mouth without panic.

Source: Companion to `gate/app.py`, `workers`, `Prefinality`

Goal

Open a Gate file, find the law, and say what must be true. This is map-reading, not becoming a full implementer overnight.

Tiny glossary

Function: A named block of instructions. Starts with `def name(...):` in Python.

Call: Using a function - asking it to run.

Return: What the function gives back (JSON, True/False, an error).

HTTP: How browsers and servers talk. GET reads. POST submits/changes.

JSON: Structured text computers exchange - keys and values in braces.

Endpoint / route: A URL path like `/v1/act` or `/diligence` that runs specific code.

Fail-closed: If unsure or broken, DENY / halt - never silent yes.

Environment variable: Secret/config outside code (API keys, Stripe, public URL).

How to read a Python function

Example shape:

```
def run_welded_act(fuse_id: str, action: str):
    """Clearance permit only. Gate never executes the write here."""
    # 1) ask Velaru if fuse is LIVE
    # 2) if DEAD or error -> DENY
    # 3) return permit with write_executed = false
```

- First line = name + inputs
- Docstring (quotes under def) = human law - read it first
- Body = steps. Skim for DENY, LIVE, CHARGE, `write_executed`
- You are hunting invariants, not memorizing every line

What to hunt in Gate files

`write_executed`

Must stay false on Gate clearance paths. If you see true, ask: did the exclusive edge actually run the write?

`fail_closed` / `timeout` / `5xx`

Broken link to Velaru must not become LIVE. Uncertainty => halt.

CHARGE

Only path DEAD -> LIVE. No admin resurrection.

ticket / fingerprint

A LIVE hop is not a blank check. Spend is married to one specific write.

their_production

Honesty flag. false until real exclusive weld.

Minimal HTTP map

- GET /diligence - human page for the money face
- GET /diligence/offer.json - machine offer
- POST /v1/act - clearance ask (not the wire)
- POST /demo/hop - public drill
- https://velaru.xyz/verify - stranger proof

How to study a file (15 minutes)

- Open the PDF or source for that topic
- Write the law in your notebook before scrolling code
- Find the function name in app.py or the worker
- Read docstring + search the keywords above
- Close it. Retrieve: what must never happen?

You are not behind

Founder-deep starts with laws. Implementer-deep comes with reps. Friday cash machine does not require you to write Cloudflare workers tonight. It requires you to not lie about what the mouth is.

Next code targets (after 01-04)

- gate/app.py - fail_closed, velaru_fuse, run_welded_act
- gate/cloudflare-worker.js - exclusive write edge
- gate/ticket.py + spend_protocol.py - married write
- gate/prefinality.py - evaluate / verify